

Efficient Verification of LTL via Proof Certificates

Verifying Linear Temporal Logic (LTL) specifications of discrete-time dynamical systems with continuous state spaces is challenging. Existing abstraction-free methods suffer from either conservatism or high computational cost. We propose proof certificates for LTL verification. Our approach transforms LTL verification into proving that accepting transitions occur finitely often. The transition persistence certificate employs ranking functions to prove accepting transitions occur finitely often. The transition safety certificate blocks reachability to accepting transitions. Combining these certificates reduces the search space while reducing conservatism by allowing systems where accepting transitions are reachable but visited finitely often. We present a synthesis method based on Satisfiability Modulo Theories (SMT). Case studies on Kuramoto oscillators and temperature control show significant speedup.

ACM Reference Format:

. 2018. Efficient Verification of LTL via Proof Certificates. *Proc. ACM Meas. Anal. Comput. Syst.* 37, 4, Article 111 (August 2018), 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Discrete-time dynamical systems are important mathematical models used to describe the characteristics of state changes in control systems. While these models can effectively capture system behavior, their infinite state space presents challenges for verification. Current methods for verifying temporal properties of these systems include the state-triplet method [24] and closure certificates [16]. However, these methods introduce new concerns: (i) The state-triplet method suffers from conservatism; (ii) The closure certificates have low synthesis efficiency. Developing a more efficient approach that reduces conservatism for verifying temporal properties in these systems remains a significant challenge.

Prior research has focused on formal verification of temporal properties—including safety, reachability, and reach-avoid specifications—within dynamical systems [2, 17, 25–27]. However, specifications for complex systems often require Linear Temporal Logic (LTL) [18] to capture temporal behaviors. Verifying LTL properties in control systems introduces several challenges. First, these properties describe behaviors across infinite execution trajectories [3], typically modeled via ω -automata. When employing proof certificates for verification, the state-triplet method requires unrolling all simple paths and blocking each individually. It is non-trivial to determine the number of certificates needed for synthesis and this method is conservative. Alternatively, closure certificates face the issue of enormous search spaces, as certificates are defined on $X \times Q \times X \times Q$. Finally, the infinite state space and nonlinear dynamics of control systems pose major obstacles for traditional verification methods.

In this work, we consider the formal verification of Linear Temporal Logic (LTL) properties for discrete-time dynamical systems using proof certificates. As an abstraction-free method, proof certificates have gained popularity in verification and static analysis domains [5–8]. Murali et al. [16] proposed closure certificates, proof certificates defined as real-valued functions over state pairs of dynamical systems, for persistence verification, which can be used to validate systems against LTL properties. Wongpiromsarn et al. [24] introduced the state-triplet method, which demonstrates that a system satisfies LTL properties by blocking all reachable paths. Inspired

Author's Contact Information:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2476-1249/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

by these two methods, we propose transition safety certificates and transition persistence certificates. Their combined use enables verification of LTL specifications expressed via transition-based Nondeterministic Büchi Automata (NBA)[4]. Based on these certificates, we present an automated synthesis method using Satisfiability Modulo Theories (SMT). We show that combining these certificates reduces the conservatism of state-triplet methods while requiring smaller search spaces than closure certificates. We validate the effectiveness of our synthesis approach through two case studies.

Contributions. The main contributions of this paper are:

- We propose *transition persistence certificates* and *transition safety certificates* for verifying LTL properties of discrete-time systems with continuous state spaces.
- We present an SMT-based synthesis method, achieving 141× to 909× speedup over closure certificates while reducing conservatism compared to state-triplet methods.

The organization of this paper is as follows: Section 2 covers preliminary knowledge. Section 3 introduces our proof certificates for LTL verification and compares with other methods. Section 4 presents the synthesis method. Section 5 demonstrates the effectiveness through case studies. Section 6 concludes the paper.

Related works: Verification of temporal logic properties via proof certificates has seen significant advancements. To validate discrete-time systems against LTL specifications, Wongpiromsarn et al. [24] developed a state-triplet method that uses barrier certificates to block all simple paths from initial states to accepting states in the automaton, requiring enumeration of state triplets (q_m, q'_m, q''_m) along these paths. Murali et al. [16] introduced closure certificates for persistence verification, which establish disjunctively well-founded transition invariants [19] for dynamical systems. Additionally, substantial research has addressed the verification of specific temporal behaviors such as reachability, safety, and avoidance[5–8]. Converting LTL specifications into Transition-based Generalized Büchi Automata (TGBA) typically yields smaller automata compared to Labeled Generalized Büchi Automata (LGBA) [9]. The degeneralization process [13] can convert a TGBA with multiple acceptance conditions into a Transition-based Büchi Automaton (TBA) featuring a single acceptance condition. Various tools exist for generating different automaton types from LTL formulas, such as Owl [15] and SPOT [11].

2 Preliminaries

2.1 Notation

We use $\mathbb{N}$ and $\mathbb{R}$ to represent the set of natural numbers and the set of real numbers, respectively. We define:

$$\begin{aligned}\mathbb{N}_{\sim i} &= \{n \in \mathbb{N} \mid n \sim i\}, \quad \text{where } i \in \mathbb{N}, \sim \in \{\geq, \leq, >, <\}, \\ \mathbb{R}_{\sim i} &= \{r \in \mathbb{R} \mid r \sim i\}, \quad \text{where } i \in \mathbb{R}, \sim \in \{\geq, \leq, >, <\}.\end{aligned}$$

Given a set S and a set S_1 , we define an indicator function:

$$I_S(x) = \begin{cases} 1, & \text{if } x \in S \\ 0, & \text{if } x \notin S \end{cases}$$

$|S|$ denotes the cardinality of S and $S \setminus S_1$ denotes $\{x \in S \mid x \notin S_1\}$.

Given a sequence $s = \langle s_0, s_1, \dots \rangle$, where $s_i \in S$ for $i \in \mathbb{N}$:

- $s[i]$ denotes s_i ;
- $s[i : j]$ denotes $\langle s_i, \dots, s_j \rangle$ for $j \geq i$;
- $s[i : \infty]$ denotes $\langle s_i, s_{i+1}, \dots \rangle$.

We say s visits S if there exists $i \in \mathbb{N}$ such that $s_i \in S$. $\text{Inf}(s)$ denotes the set of elements that appear in s infinitely often. S^ω denotes the set of infinite sequences over S , i.e., $S^\omega = \{s \mid s = \langle s_0, s_1, \dots \rangle \text{ and } s_i \in S \text{ for } i \in \mathbb{N}\}$. A function $f : S \rightarrow \mathbb{R}$ is bounded if and only if there exist $a, b \in \mathbb{R}$ such that $a \leq f(x) \leq b$ for all $x \in S$.

2.2 Discrete-time Dynamical System

A discrete-time dynamical system $\mathfrak{S}$ is a tuple (X, X_0, f) , where $X \subseteq \mathbb{R}^n$ represents system states, $X_0 \subseteq X$ represents initial states, and $f : X \rightarrow X$ represents the transition function. The evolution of the system state is described as follows:

$$\mathfrak{S} : x_{t+1} = f(x_t)$$

The state space is compact. An infinite sequence of states $\langle x_0, x_1, x_2, \dots \rangle$ is called a *trajectory* of the system if $x_0 \in X_0$ and $x_{i+1} = f(x_i)$ for $i \in \mathbb{N}$. We use $\mathfrak{T}(\mathfrak{S})$ to represent all *trajectories* on $\mathfrak{S}$. The *predecessor* of x_{i+1} is x_i for $i \in \mathbb{N}$. In a given *trajectory*, x_{i+1} has only one *predecessor*. We use the notation x_{i+1}^{pre} to denote the *predecessor* of x_{i+1} in a *trajectory*.

2.3 Specification

This paper focuses on safety, persistence, and LTL properties of discrete-time dynamical systems.

Safety: A system $\mathfrak{S} = (X, X_0, f)$ is *safe* with respect to unsafe states $X_u \subseteq X$ if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we have $x_i \notin X_u$ for all $i \in \mathbb{N}$.

Persistence: A system $\mathfrak{S} = (X, X_0, f)$ is *persistent* with respect to states $X_{VF} \subseteq X$ if for any *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, elements of X_{VF} appear finitely often in τ .

Linear Temporal Logic (LTL): LTL formulae [18] are defined over a set of atomic propositions AP . A labelling function $\mathfrak{L} : X \rightarrow 2^{AP}$ maps system states to subsets of AP . Given a *trajectory* $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, we denote $\mathfrak{L}(\tau) = \langle \mathfrak{L}(x_0), \mathfrak{L}(x_1), \dots \rangle$ as the corresponding *word*.

The syntax of LTL is:

$$\phi := \top \mid a \mid \neg\phi \mid \phi \wedge \phi \mid X\phi \mid \phi U \phi$$

where $a \in AP$ is an atomic proposition. Temporal operators finally (F) and always (G) are derived from next (X) and until (U).

Given a *word* $w = \mathfrak{L}(\tau)$ and an LTL formula ϕ , the semantics is defined inductively:

- $w \models \top$ always holds
- $w \models a$ iff $a \in w[0]$
- $w \models \phi_1 \wedge \phi_2$ iff $w \models \phi_1$ and $w \models \phi_2$
- $w \models \neg\phi_1$ iff $w \not\models \phi_1$
- $w \models X\phi_1$ iff $w[1 : \infty] \models \phi_1$
- $w \models \phi_1 U \phi_2$ iff there exists $j \in \mathbb{N}$ such that $w[j : \infty] \models \phi_2$ and for all $i \in \mathbb{N}_{<j}$, $w[i : \infty] \models \phi_1$

We say $\mathfrak{S} \models_{\mathfrak{L}} \phi$ if for any $\tau \in \mathfrak{T}(\mathfrak{S})$, we have $\mathfrak{L}(\tau) \models \phi$.

Nondeterministic Büchi Automaton (NBA): An NBA is a tuple $(\Sigma, Q, q_0, \delta, Acc)$, where $\Sigma = 2^{AP}$ is the alphabet, Q is a finite set of states, $q_0 \in Q$ is the initial state, $\delta \subseteq Q \times \Sigma \times Q$ is the transition relation, and $Acc \subseteq \delta$ is the set of *accepting transitions*. Given a word $w = \langle w_0, w_1, \dots \rangle \in \Sigma^\omega$, a **run** on w is a sequence $r = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \dots \rangle$ where $r_0 = q_0$ and $(r_i, w_i, r_{i+1}) \in \delta$ for all $i \in \mathbb{N}$. An NBA accepts w if there exists a run where accepting transitions occur infinitely often, i.e., $Inf(r) \cap Acc \neq \emptyset$.

For an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, we define:

$$Acc^{pair} = \{(p, q) \mid \exists \sigma \in \Sigma. (p, \sigma, q) \in Acc\}, \quad (1)$$

$$Acc^{start} = \{p \mid \exists \sigma \in \Sigma, q \in Q. (p, \sigma, q) \in Acc\}, \quad (2)$$

$$Acc^{k,l} = \{(q_k, \sigma, q_l) \mid (q_k, \sigma, q_l) \in Acc\}. \quad (3)$$

We denote $\mathfrak{L}(\mathfrak{S}) = \{\mathfrak{L}(\tau) \mid \tau \in \mathfrak{T}(\mathfrak{S})\}$ as the *language* of $\mathfrak{S}$, and $\mathfrak{L}(\mathcal{A})$ as the language accepted by $\mathcal{A}$. To verify $\mathfrak{S} \models_{\mathfrak{L}} \phi$, we construct an NBA $\mathcal{A}$ for $\neg\phi$ and show $\mathfrak{L}(\mathfrak{S}) \cap \mathfrak{L}(\mathcal{A}) = \emptyset$.

2.4 Related Certificates

Barrier certificates and closure certificates are used to verify safety [20], persistence [16], and LTL properties [16], respectively. We provide their definitions and related conclusions from prior work.

Definition 2.1 (barrier certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ and $\mathcal{X}_u \subseteq \mathcal{X}$, a bounded function $B(x) : \mathcal{X} \rightarrow \mathbb{R}$ is a *barrier certificate* with respect to $\mathcal{X}_u$ such that $x_0 \in \mathcal{X}_0$, $x_u \in \mathcal{X}_u$, $x \in \mathcal{X}$ and $x' = f(x)$. We have:

$$B(x_0) \geq 0, \quad (4)$$

$$B(x_u) < 0 \quad (5)$$

$$B(x) \geq 0 \Rightarrow B(x') \geq 0. \quad (6)$$

THEOREM 2.2 (SAFETY). *If a barrier certificate $B(x)$ can be found such that it ensures all trajectories in $\mathfrak{T}(\mathfrak{S})$ cannot visit $\mathcal{X}_u$, then the system $\mathfrak{S}$ is safe with respect to $\mathcal{X}_u$.*

Definition 2.3 (closure certificate). Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow \Sigma$ and an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, \text{Acc})$ representing the negation of an LTL formula ϕ . A bounded function $T : \mathcal{X} \times Q \times \mathcal{X} \times Q \rightarrow \mathbb{R}$ is a closure certificate for ϕ if there exists a value $\epsilon \in \mathbb{R}_{>0}$ such that for all states $x, y, y' \in \mathcal{X}$, $x' = f(x)$, states $q_i, q_j \in Q$, and $q'_i \in \delta(q_i, \mathfrak{L}(x))$, we have:

$$T((x, q_i), (x', q'_i)) \geq 0 \quad (7)$$

$$T((x', q'_i), (y, q_j)) \geq 0 \Rightarrow T((x, q_i), (y, q_j)) \geq 0 \quad (8)$$

and for all states $x_0 \in \mathcal{X}_0$, and $\ell, \ell' \in \text{Acc}$, we have:

$$\begin{aligned} T((x_0, q_0), (y, \ell)) &\geq 0 \wedge T((y, \ell), (y', \ell')) \geq 0 \\ \Rightarrow T((x_0, q_0), (y', \ell')) &\leq T((x_0, q_0), (y, \ell)) - \epsilon. \end{aligned} \quad (9)$$

THEOREM 2.4 (LTL). *Consider a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a set of atomic propositions AP , a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$ and an LTL formula ϕ . Let an NBA $\mathcal{A} = (2^{AP}, Q, q_0, \delta, \text{Acc})$ represent $\neg\phi$. The existence of a closure certificate satisfying conditions (7)-(9) implies that $\mathfrak{S} \models_{\mathfrak{L}} \phi$.*

2.5 Problem Statement

Problem. Given a discrete-time dynamical system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labeling function $\mathfrak{L} : \mathcal{X} \rightarrow 2^{AP}$, and an LTL formula ϕ over atomic propositions AP , verify that $\mathfrak{S} \models_{\mathfrak{L}} \phi$ holds.

3 Proof Certificates

We propose proof certificates for LTL verification: *transition safety certificates* and *transition persistence certificates*. To verify that a system $\mathfrak{S}$ satisfies an LTL formula ϕ , we transform the verification problem into proving that accepting transitions in the NBA for $\neg\phi$ occur finitely often. By the definition of NBA acceptance, a word is accepted if and only if accepting transitions occur infinitely often along some run. If we prove all accepting transitions occur only finitely often for any trajectory in $\mathfrak{T}(\mathfrak{S})$, then no word in $\mathfrak{L}(\mathfrak{S})$ can be accepted by the NBA for $\neg\phi$, thereby establishing $\mathfrak{S} \models_{\mathfrak{L}} \phi$. Both certificates ensure that accepting transitions in the NBA (representing $\neg\phi$) occur only finitely often. The safety certificate proves accepting transitions never occur by blocking reachability to their start states. The persistence certificate allows such states to be reachable but proves transitions occur finitely often via a ranking function.

3.1 Transition Safety Certificate

We prove accepting transitions never occur by showing their start states are unreachable in the product system.

Definition 3.1 (transition safety certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow \Sigma$ and an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, \text{Acc})$. Construct the product system $\mathfrak{S} \times \mathcal{A} = (\mathcal{Y}, \mathcal{Y}_0, F)$, where $\mathcal{Y} = \{(x, q) \mid x \in \mathcal{X}, q \in Q\}$, $\mathcal{Y}_0 = \{(x_0, q_0) \mid x_0 \in \mathcal{X}_0\}$, and $F(x, q) = \{(x', q') \mid x' = f(x) \wedge (q, \mathfrak{L}(x), q') \in \delta\}$. For $q_k \in \text{Acc}^{\text{start}}$, define the unsafe set $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$.

A bounded function $B_k(x, q) : \mathcal{X} \times Q \rightarrow \mathbb{R}$ is a *transition safety certificate* with respect to $q_k \in \text{Acc}^{\text{start}}$ if for all $(x, q) \in \mathcal{Y}$, $(x_0, q_0) \in \mathcal{Y}_0$, $(x', q') \in F(x, q)$, we have:

$$B_k(x_0, q_0) \geq 0, \quad (10)$$

$$B_k(x, q_k) < 0, \quad (11)$$

$$B_k(x, q) \geq 0 \Rightarrow B_k(x', q') \geq 0. \quad (12)$$

The certificate B_k forms a barrier in the product system: non-negative at initial states (10), negative at all (x, q_k) (11), and invariant under transitions (12). This guarantees (x, q_k) is unreachable, preventing all accepting transitions from q_k .

LEMMA 3.2 (TRANSITION SAFETY). *If a transition safety certificate B_k with respect to $q_k \in \text{Acc}^{\text{start}}$ can be found, then all runs of words in $\mathfrak{L}(\mathfrak{S})$ never visit state q_k , thereby preventing all accepting transitions starting from q_k .*

3.2 Transition Persistence Certificate

We prove accepting transitions occur finitely often using a ranking function that decreases with each occurrence.

Definition 3.3 (transition persistence certificate). Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow \Sigma$ and an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, \text{Acc})$. For $(k, l) \in \text{Acc}^{\text{pair}}$, define the indicator function $I_{k,l}(x) = 1$ if $(q_k, \mathfrak{L}(x), q_l) \in \text{Acc}^{k,l}$, and 0 otherwise.

A bounded function $B_{k,l}(x) : \mathcal{X} \rightarrow \mathbb{R}$ is a *transition persistence certificate* with respect to $\text{Acc}^{k,l}$ if there exists $\epsilon > 0$ such that for all $x_0 \in \mathcal{X}_0$, $x \in \mathcal{X}$, $x' = f(x)$, we have:

$$B_{k,l}(x_0) \geq 0, \quad (13)$$

$$B_{k,l}(x) \geq 0 \Rightarrow B_{k,l}(x') \geq 0, \quad (14)$$

$$B_{k,l}(x) \geq B_{k,l}(x') + I_{k,l}(x)\epsilon. \quad (15)$$

The certificate $B_{k,l}$ is non-negative initially (13), remains non-negative along trajectories (14), and decreases by ϵ whenever the state's label matches an accepting transition from q_k to q_l (15). By (13) and (14), $B_{k,l}$ remains non-negative along any trajectory starting from $x_0 \in \mathcal{X}_0$. By (15), $B_{k,l}$ decreases by at least ϵ each time $I_{k,l}(x) = 1$. Since accepting transitions from q_k to q_l can only occur when $(q_k, \mathfrak{L}(x), q_l) \in \text{Acc}^{k,l}$, the number of such transitions is bounded by $B_{k,l}(x_0)/\epsilon$, ensuring finite occurrence.

LEMMA 3.4 (TRANSITION PERSISTENCE). *If a transition persistence certificate $B_{k,l}$ can be found, then all runs of words in $\mathfrak{L}(\mathfrak{S})$ make accepting transitions from q_k to q_l happen finitely often.*

3.3 Main Verification Theorem

We now present the main theorem for LTL verification. For each accepting transition start state in the NBA, we use either a transition safety certificate or transition persistence certificates to prove accepting transitions from that state occur finitely often.

THEOREM 3.5 (LTL VERIFICATION). *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow \Sigma$ and an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, \text{Acc})$ representing the negation of an LTL formula ϕ . For each $q_k \in \text{Acc}^{\text{start}}$, if we can find either:*

- (1) A transition safety certificate B_k with respect to q_k , or
- (2) Transition persistence certificates $B_{k,l}$ for all $(k, l) \in Acc^{pair}$ where q_k is the start state,

then it can be guaranteed that $\mathfrak{S} \models_{\mathcal{L}} \phi$.

PROOF. Given an accepting transition start state $q_k \in Acc^{start}$:

- (1) If a transition safety certificate B_k is found, by Lemma (A.1), all runs never visit state q_k , thereby preventing all accepting transitions starting from q_k .
- (2) For each $(k, l) \in Acc^{pair}$ where the start state is q_k , if a transition persistence certificate $B_{k,l}$ is found, by Lemma (A.2), all runs make accepting transitions from q_k to q_l happen finitely often.

This ensures that all accepting transitions in Acc occur only finitely often. By the definition of NBA acceptance, a word w is accepted if and only if there exists a run where accepting transitions occur infinitely often, i.e., $Inf(r) \cap Acc \neq \emptyset$. Since all accepting transitions occur only finitely often, for any word $w \in \mathcal{L}(\mathfrak{S})$ and any run r on w , we have $Inf(r) \cap Acc = \emptyset$. Therefore, no word in $\mathcal{L}(\mathfrak{S})$ is accepted by automaton $\mathcal{A}$, implying $\mathcal{L}(\mathfrak{S}) \cap \mathcal{L}(\mathcal{A}) = \emptyset$. Since $\mathcal{L}(\mathcal{A}) = \mathcal{L}(\neg\phi)$, we have $\mathcal{L}(\mathfrak{S}) \subseteq \mathcal{L}(\phi)$, and consequently $\mathfrak{S} \models_{\mathcal{L}} \phi$. $\square$

The two certificates are complementary: safety certificates block reachability to prove transitions never occur, while persistence certificates allow reachability but prove finite occurrence via ranking functions. This combination reduces conservatism while maintaining smaller search spaces than closure certificates. To illustrate their complementarity, consider verifying $G\neg a$ with the NBA for Fa (Figure 1), where accepting transitions $(q_0, \emptyset, q_0)$ and $(q_0, \{a\}, q_0)$ form a self-loop. For the persistence certificate $B_{0,0}$, conditions (13-15) require $B_{0,0}(x_0) \geq 0$, invariance under transitions, and $B_{0,0}(x) \geq B_{0,0}(x') + \epsilon$. In this self-loop, the accepting transition occurs at every step, forcing infinite descent—contradicting boundedness. The persistence certificate fails for infinitely-occurring transitions. However, a safety certificate can prove q_0 is unreachable in the product system. Conversely, if q_0 is reachable but accepting transitions occur finitely often, the persistence certificate succeeds. This demonstrates their complementary strengths.

3.4 Compared with Other Methods

In the context of this discussion, we assume that the NBA for an LTL specification ϕ has already been constructed. The translation from an LTL formula to an NBA involves an exponential complexity with respect to the size of the formula [23]. The complexity of constructing the complement of an NBA is known to be EXPTIME [21].

The State-Triplet Approach. The state-triplet approach [24] uses barrier certificates (2.1) to separate safe regions X_{safe} from unsafe regions X_u . It blocks all simple paths from q_0 to accepting states in the NBA for $\neg\phi$. Finding certificates for all such paths ensures $\mathcal{L}(\mathfrak{S})$ never visits accepting states, verifying $\mathfrak{S} \models_{\mathcal{L}} \phi$. Our transition safety certificate shares this reachability-blocking principle but operates on the product system $\mathfrak{S} \times \mathcal{A}$, eliminating the need to enumerate simple paths. The state-triplet method requires one certificate per triplet to block each path individually, while we synthesize one certificate per accepting transition start state. Our persistence certificate further allows visiting accepting states while proving transitions occur finitely often. Combining both certificates reduces conservatism: beyond blocking reachability, we verify systems where runs reach accepting states yet satisfy the specification.

Closure Certificates. Closure certificates (2.3) are defined on $X \times Q \times X \times Q$, creating high-dimensional search spaces with heavy computational burdens. Our certificates operate on smaller spaces: persistence certificates on X , safety certificates on $X \times Q$. We require $|Acc^{pair}|$ persistence certificates and $|Acc^{start}|$ safety certificates, yielding a search space of at most $X \times Q \times Q$ —smaller than $X \times Q \times X \times Q$. Case studies show that for benchmarks requiring closure certificates [16], our certificates synthesize in seconds with significant speedup.

4 Certificate Synthesis

In practice, we must synthesize certificates automatically. This section presents an SMT-based method using counterexample-guided inductive synthesis. We apply counterexample-guided inductive synthesis (CEGIS) [22] to automatically construct both transition safety certificates and transition persistence certificates. The core idea is to iteratively refine candidate certificates by sampling states and using SMT solvers to satisfy certificate conditions on these samples.

Transition Safety Certificates. Given an accepting state $q_k \in Acc^{start}$ in NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, we define a parametric template:

$$B_k(x, q; \mathbf{c}) = \sum_{i=1}^m c_i p_i(x, q), \quad (16)$$

where $\{p_1, \dots, p_m\}$ is a user-defined basis of monomials over $\mathcal{X} \times \mathcal{Q}$ with degree at most d , and $\mathbf{c} = (c_1, \dots, c_m) \in \mathbb{R}^m$ are unknown coefficients. Our goal is to find $\mathbf{c}$ such that $B_k(\cdot, \cdot; \mathbf{c})$ satisfies conditions (10), (11), and (12). Sample finite sets $S_0 \subseteq \mathcal{X}_0 \times \{q_0\}$, $S_{\mathcal{X}} \subseteq \mathcal{X} \times \mathcal{Q}$, and $S_u \subseteq \mathcal{X} \times \{q_k\}$. The SMT formula for candidate generation is:

$$\Phi_{\text{sample}} \triangleq \bigwedge_{(x,q) \in S_0} B_k(x, q; \mathbf{c}) \geq 0 \quad (17)$$

$$\wedge \bigwedge_{\substack{(x,q) \in S_{\mathcal{X}} \\ (q, \mathfrak{L}(x), q') \in \delta}} [B_k(x, q; \mathbf{c}) \geq 0 \Rightarrow B_k(f(x), q'; \mathbf{c}) \geq 0] \quad (18)$$

$$\wedge \bigwedge_{(x,q) \in S_u} B_k(x, q; \mathbf{c}) < 0. \quad (19)$$

If satisfiable with solution $\hat{\mathbf{c}}$, define $\hat{B}_k = B_k(\cdot, \cdot; \hat{\mathbf{c}})$ and verify by checking:

$$\Phi_{\text{init}}^- \triangleq x \in \mathcal{X}_0 \wedge \hat{B}_k(x, q_0) < 0, \quad (20)$$

$$\Phi_{\text{inv}}^- \triangleq x \in \mathcal{X} \wedge (q, \mathfrak{L}(x), q') \in \delta \wedge \hat{B}_k(x, q) \geq 0 \wedge \hat{B}_k(f(x), q') < 0, \quad (21)$$

$$\Phi_{\text{unsafe}}^- \triangleq x \in \mathcal{X} \wedge \hat{B}_k(x, q_k) \geq 0. \quad (22)$$

Counterexamples are added to S_0 , $S_{\mathcal{X}}$, or S_u accordingly and the process iterates until no counterexamples are found, yielding a valid transition safety certificate.

Transition Persistence Certificates. For each accepting transition pair $(q_k, q_l) \in Acc^{pair}$, we define a template:

$$B_{k,l}(x; \mathbf{c}) = \sum_{i=1}^{m'} c_i p_i(x), \quad (23)$$

where $\{p_1, \dots, p_{m'}\}$ are monomials over $\mathcal{X}$ and $\mathbf{c} \in \mathbb{R}^{m'}$. We seek $\mathbf{c}$ and $\epsilon > 0$ satisfying conditions (13), (14), and (15). Sample finite sets $S_0 \subseteq \mathcal{X}_0$ and $S_{\mathcal{X}} \subseteq \mathcal{X}$. The SMT formula for candidate generation is:

$$\Psi_{\text{sample}} \triangleq \bigwedge_{x \in S_0} B_{k,l}(x; \mathbf{c}) \geq 0 \quad (24)$$

$$\wedge \bigwedge_{x \in S_{\mathcal{X}}} [B_{k,l}(x; \mathbf{c}) \geq 0 \Rightarrow B_{k,l}(f(x); \mathbf{c}) \geq 0] \quad (25)$$

$$\wedge \bigwedge_{\substack{x \in S_{\mathcal{X}} \\ (q_k, \mathfrak{Q}(x), q_l) \notin \text{Acc}^{k,l}}} B_{k,l}(x; \mathbf{c}) \geq B_{k,l}(f(x); \mathbf{c}) \quad (26)$$

$$\wedge \bigwedge_{\substack{x \in S_{\mathcal{X}} \\ (q_k, \mathfrak{Q}(x), q_l) \in \text{Acc}^{k,l}}} B_{k,l}(x; \mathbf{c}) \geq B_{k,l}(f(x); \mathbf{c}) + \epsilon. \quad (27)$$

If satisfiable with solution $(\hat{\mathbf{c}}, \hat{\epsilon})$, define $\hat{B}_{k,l} = B_{k,l}(\cdot; \hat{\mathbf{c}})$ and verify by checking:

$$\Psi_{\text{init}}^- \triangleq x \in \mathcal{X}_0 \wedge \hat{B}_{k,l}(x) < 0, \quad (28)$$

$$\Psi_{\text{inv}}^- \triangleq x \in \mathcal{X} \wedge \hat{B}_{k,l}(x) \geq 0 \wedge \hat{B}_{k,l}(f(x)) < 0, \quad (29)$$

$$\Psi_{\text{dec}}^- \triangleq x \in \mathcal{X} \wedge (q_k, \mathfrak{Q}(x), q_l) \in \text{Acc}^{k,l} \wedge \hat{B}_{k,l}(x) < \hat{B}_{k,l}(f(x)) + \hat{\epsilon}, \quad (30)$$

$$\Psi_{\text{non-dec}}^- \triangleq x \in \mathcal{X} \wedge (q_k, \mathfrak{Q}(x), q_l) \notin \text{Acc}^{k,l} \wedge \hat{B}_{k,l}(x) < \hat{B}_{k,l}(f(x)). \quad (31)$$

Counterexamples are added to samples and the process iterates until no counterexamples are found, yielding a valid transition persistence certificate.

5 Experimental Evaluation

We evaluate our certificate-based verification framework on two benchmark systems to demonstrate: (1) the effectiveness of transition safety and persistence certificates for LTL verification, (2) computational efficiency compared to prior closure certificate methods [16], and (3) the practical applicability of SMT-based synthesis. All experiments run on Ubuntu 22.04.3 LTS (Intel i7-9750H, 8 GB RAM). We use Z3 [10] for linear arithmetic and dReal [12] ($\delta = 0.0001$) for nonlinear verification. Table 1 compares synthesis times. Our certificates achieve **141× to 909× speedup** over closure certificates [16], confirming the efficiency gains from reduced search space complexity.

Table 1. Synthesis time comparison (seconds).

Benchmark	Our Method	Closure [16]	Speedup
Kuramoto 1D	4.25	600	141×
Kuramoto 2D	7.27	6600	909×
Two-Room	8.77	5400	616×

5.1 Kuramoto Oscillator: Safety Verification

The Kuramoto model describes coupled oscillators in neuroscience and power systems [14]. We verify safety specifications $G \neg a$ (always avoid unsafe states) for one-dimensional and two-dimensional scenarios.

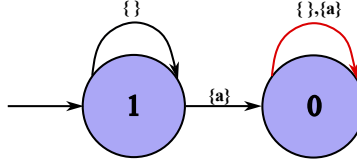


Fig. 1. NBA representing Fa (negation of safety specification). Accepting transitions (red) form a self-loop at q_0 .

5.1.1 One-Dimensional System. The system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$ has state space $\mathcal{X} = [0, 2\pi]$, initial states $\mathcal{X}_0 = [\frac{4\pi}{9}, \frac{5\pi}{9}]$, unsafe states $\mathcal{X}_u = [\frac{7\pi}{9}, \frac{8\pi}{9}]$, and dynamics:

$$f(x) = x + t_s \Omega + t_s K \sin(-x) - 0.532x^2 + 1.69, \quad (32)$$

where $t_s = 0.1$ (sampling time), $\Omega = 0.01$ (natural frequency), $K = 0.0006$ (coupling strength).

With $AP = \{a\}$ and labeling $\mathfrak{L}(x) = \{a\}$ iff $x \in \mathcal{X}_u$, safety is expressed as $\mathbf{G}\neg a$. The negation Fa is represented by the NBA in Figure 1 with accepting transitions $(q_0, \{\}, q_0)$ and $(q_0, \{a\}, q_0)$ starting from $q_0 \in Acc^{start}$. We synthesize a transition safety certificate $B_0(x, q)$ using CEGIS with polynomial template:

$$B_0(x, q; \mathbf{c}) = \sum_{i=0}^4 c_i x^i + \sum_{j=1}^3 c_{4+j} q^j + c_8 xq. \quad (33)$$

Sampling 629 states from $\mathcal{X}$ and 35 from $\mathcal{X}_0$, Z3 generates candidates and dReal verifies counterexamples. The synthesized certificate is:

$$B_0(x, q) = -1 + 2q^2 - \frac{7}{16}xq. \quad (34)$$

CEGIS converges in **4.25 seconds**, proving safety. In contrast, closure certificate synthesis [16] requires **10 minutes**—a **141× speedup**.

5.1.2 Two-Dimensional System. The system extends to $\mathcal{X} = [0, \frac{8\pi}{9}]^2$, $\mathcal{X}_0 = [0, \frac{\pi}{9}]^2$, $\mathcal{X}_u = ([0, \frac{8\pi}{9}] \times [\frac{5\pi}{6}, \frac{8\pi}{9}]) \cup ([\frac{5\pi}{6}, \frac{8\pi}{9}] \times [0, \frac{8\pi}{9}])$, with dynamics:

$$f(x_1, x_2) = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + t_s \begin{bmatrix} \Omega + 1.69 \\ \Omega + 1.69 \end{bmatrix} + K t_s \begin{bmatrix} \sin(x_2 - x_1) \\ \sin(x_1 - x_2) \end{bmatrix} - 0.532 t_s \begin{bmatrix} x_1^2 \\ x_2^2 \end{bmatrix}. \quad (35)$$

The labeling marks $\mathfrak{L}(x_1, x_2) = \{a\}$ when $x_1 \in [\frac{5\pi}{6}, \frac{8\pi}{9}]$ or $x_2 \in [\frac{5\pi}{6}, \frac{8\pi}{9}]$, using the same NBA from Figure 1. CEGIS with template $B_0(x_1, x_2, q; \mathbf{c}) = c_0 + c_1 x_1 + c_2 x_2^2 + c_3 x_1 x_2 + c_4 x_1^3 + c_5 q + c_6 q^2 + c_7 q x_1^2 + c_8 x_2 q$ yields:

$$B_0(x_1, x_2, q) = -1 + \frac{1}{8}x_1 x_2 + \frac{3}{2}q^2 - \frac{3}{32}q x_1^2 - \frac{153}{512}x_2 q. \quad (36)$$

CEGIS completes in **7.27 seconds**. The closure method requires **110 minutes** [16]—a **909× speedup**.

5.2 Two-Room Temperature: Persistence Verification

We verify an interconnected temperature control system [1] where rooms must visit a comfort zone only finitely often from initial states. State space $\mathcal{X} = [20, 34]^2$ represents room temperatures, $\mathcal{X}_0 = [21, 24]^2$ are initial states, and:

$$f(x_1, x_2) = A \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \mu T_h \begin{bmatrix} u(x_1) \\ u(x_2) \end{bmatrix} + \theta \begin{bmatrix} T_e \\ T_e \end{bmatrix}, \quad (37)$$

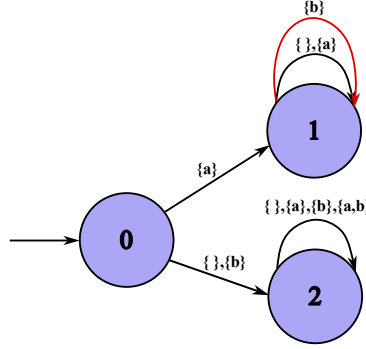


Fig. 2. NBA for $a \wedge \text{GF}b$ (negation of $a \Rightarrow \text{FG}\neg b$). Accepting transitions (red) are $(q_1, \{b\}, q_1)$ and $(q_1, \{a, b\}, q_1)$.

where

$$A = \begin{bmatrix} 1 - 2\alpha - \theta - \mu u(x_1) & \alpha \\ \alpha & 1 - 2\alpha - \theta - \mu u(x_2) \end{bmatrix}, \quad (38)$$

with parameters $\alpha = 0.004$, $\theta = 0.01$, $\mu = 0.15$, $u(x_i) = 0.59 - 0.011x_i$, $T_h = 40$, $T_e = 0$.

With $AP = \{a, b\}$, label $\mathfrak{L}(x_1, x_2) = \{a\}$ for $(x_1, x_2) \in \mathcal{X}_0$, $\mathfrak{L}(x_1, x_2) \ni b$ for $(x_1, x_2) \in [20, 26]^2$. The specification $a \Rightarrow \text{FG}\neg b$ (from initial states, eventually stay outside comfort zone forever) has negation $a \wedge \text{GF}b$ represented by the NBA in Figure 2 with accepting transitions from q_1 . First, we attempt transition safety synthesis to prove q_1 is unreachable. This **fails** after 16.24 seconds, indicating q_1 may be reachable. We then synthesize transition persistence certificates to prove accepting transitions occur finitely often.

Using SMT-based CEGIS with template $B_{1,1}(x_1, x_2; \mathbf{c}) = c_0 + \sum_{i=1,2} (c_i I_{\mathcal{X}_0} + c_{i+2} I_{[20,26]^2}) x_i + c_7 \max(x_1, x_2) + c_8 x_1^2 + c_9 x_2^2$, where I_S is the indicator for set S , we obtain:

$$B_{1,1}(x_1, x_2) = 27I_{[20,26]^2}(x_1, x_2) - x_1 I_{[20,26]^2}(x_1, x_2), \quad \epsilon = 0.25. \quad (39)$$

Synthesis time: **8.77 seconds**. The closure method requires **90 minutes** [16]—a **616× speedup**.

6 Conclusion

This paper introduces transition safety certificates and transition persistence certificates for verifying LTL properties of discrete-time dynamical systems. Their complementarity reduces conservatism compared to state-triplet methods [24]. Our certificates operate on search spaces no larger than $\mathcal{X} \times \mathcal{Q} \times \mathcal{Q}$, significantly smaller than closure certificates [16]. As future work, we plan on investigating data driven approaches to find these certificates as well as investigate their use in synthesizing controllers.

References

- [1] Mahathi Anand, Abolfazl Lavaei, and Majid Zamani. 2024. Compositional synthesis of control barrier certificates for networks of stochastic systems against ω -regular specifications. *Nonlinear Analysis: Hybrid Systems* 51 (2024), 101427.
- [2] Matin Ansari-pour, Krishnendu Chatterjee, Thomas A Henzinger, Mathias Lechner, and Đorđe Žikelić. 2023. Learning provably stabilizing neural controllers for discrete-time stochastic systems. In *International Symposium on Automated Technology for Verification and Analysis*. Springer, 357–379.
- [3] Christel Baier and Joost-Pieter Katoen. 2008. *Principles of model checking*. MIT press.
- [4] J Richard Büchi. 1966. Symposium on decision problems: On a decision method in restricted second order arithmetic. In *Studies in Logic and the Foundations of Mathematics*. Vol. 44. Elsevier, 1–11.
- [5] Aleksandar Chakarov and Sriram Sankaranarayanan. 2013. Probabilistic program analysis with martingales. In *Computer Aided Verification: 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13–19, 2013. Proceedings* 25. Springer, 511–526.

- [6] Krishnendu Chatterjee, Hongfei Fu, Petr Novotný, and Rouzbeh Hasheminezhad. 2016. Algorithmic analysis of qualitative and quantitative termination problems for affine probabilistic programs. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 327–342.
- [7] Krishnendu Chatterjee, Amir Goharshady, Ehsan Goharshady, Mehrdad Karrabi, and Đorđe Žikelić. 2024. Sound and complete witnesses for template-based verification of LTL properties on polynomial programs. In *International Symposium on Formal Methods*. Springer, 600–619.
- [8] Krishnendu Chatterjee, Ehsan Kafshdar Goharshady, Petr Novotný, and Đorđe Žikelić. 2024. Equivalence and similarity refutation for probabilistic programs. *Proceedings of the ACM on Programming Languages* 8, PLDI (2024), 2098–2122.
- [9] Jean-Michel Couvreur. 1999. On-the-fly verification of linear temporal logic. In *International Symposium on Formal Methods*. Springer, 253–271.
- [10] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.
- [11] Alexandre Duret-Lutz, Alexandre Lewkowicz, Amaury Fauchille, Thibaud Michaud, Étienne Renault, and Laurent Xu. 2016. Spot 2.0 — A Framework for LTL and omega-Automata Manipulation. In *Automated Technology for Verification and Analysis*, Cyrille Artho, Axel Legay, and Doron Peled (Eds.). Springer International Publishing, Cham, 122–129.
- [12] Sicun Gao, Soonho Kong, and Edmund M Clarke. 2013. dReal: An SMT solver for nonlinear theories over the reals. In *International conference on automated deduction*. Springer, 208–214.
- [13] Dimitra Giannakopoulou and Flavio Lerda. 2002. From states to transitions: Improving translation of LTL formulae to Büchi automata. In *International Conference on Formal Techniques for Networked and Distributed Systems*. Springer, 308–326.
- [14] Yufeng Guo, Dongrui Zhang, Zhuchun Li, Qi Wang, and Daren Yu. 2021. Overviews on the applications of the Kuramoto model in modern power system analysis. *International Journal of Electrical Power & Energy Systems* 129 (2021), 106804.
- [15] Jan Křetínský, Tobias Meggendorfer, and Salomon Sickert. 2018. Owl: A Library for omega-Words, Automata, and LTL. In *Automated Technology for Verification and Analysis*, Shuvendu K. Lahiri and Chao Wang (Eds.). Springer International Publishing, Cham, 543–550.
- [16] Vishnu Murali, Ashutosh Trivedi, and Majid Zamani. 2024. Closure certificates. In *Proceedings of the 27th ACM International Conference on Hybrid Systems: Computation and Control*. 1–11.
- [17] Grigory Neustroev, Mirco Giacobbe, and Anna Lukina. 2025. Neural continuous-time supermartingale certificates. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 27538–27546.
- [18] Amir Pnueli. 1977. The temporal logic of programs. In *18th annual symposium on foundations of computer science (sfcs 1977)*. iee, 46–57.
- [19] Andreas Podelski and Andrey Rybalchenko. 2004. Transition invariants. In *Proceedings of the 19th Annual IEEE Symposium on Logic in Computer Science, 2004*. IEEE, 32–41.
- [20] Stephen Prajna and Ali Jadbabaie. 2004. Safety verification of hybrid systems using barrier certificates. In *International workshop on hybrid systems: Computation and control*. Springer, 477–492.
- [21] S. Safra. 1988. On the complexity of omega-automata. In *[Proceedings 1988] 29th Annual Symposium on Foundations of Computer Science*. 319–327. doi:10.1109/SFCS.1988.21948
- [22] Armando Solar-Lezama. 2008. *Program synthesis by sketching*. University of California, Berkeley.
- [23] Moshe Y Vardi. 2005. An automata-theoretic approach to linear temporal logic. *Logics for concurrency: structure versus automata* (2005), 238–266.
- [24] Tichakorn Wongpiromsarn, Ufuk Topcu, and Andrew Lamperski. 2016. Automata theory meets barrier certificates: Temporal logic verification of nonlinear systems. *IEEE Trans. Automat. Control* 61, 11 (2016), 3344–3355.
- [25] Bai Xue, Renjue Li, Naijun Zhan, and Martin Fränzle. 2021. Reach-avoid analysis for stochastic discrete-time systems. In *2021 American Control Conference (ACC)*. IEEE, 4879–4885.
- [26] Đorđe Žikelić, Mathias Lechner, Thomas A Henzinger, and Krishnendu Chatterjee. 2023. Learning control policies for stochastic systems with reach-avoid guarantees. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 11926–11935.
- [27] Đorđe Žikelić, Mathias Lechner, Abhinav Verma, Krishnendu Chatterjee, and Thomas Henzinger. 2023. Compositional policy learning in stochastic control systems with formal guarantees. *Advances in Neural Information Processing Systems* 36 (2023), 47849–47873.

A Proof

THEOREM A.1. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, a trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$, a labelling function $\mathfrak{L} : \mathcal{X} \rightarrow \Sigma$ and an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, \text{Acc})$, if a transition safety certificate $B_k(x, q)$ with respect to $q_k \in \text{Acc}^{\text{start}}$ can be found, then it can be guaranteed that all runs of all words in $\mathfrak{L}(\mathfrak{S})$ cannot visit the state q_k .*

PROOF. By Definition 3.1, we have $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$ as the unsafe set in the product system $\mathfrak{S} \times \mathcal{A}$. We verify that B_k satisfies the barrier certificate conditions (4, 5, 6) with respect to $\mathcal{Y}_u$:

- Condition (10) states $B_k(x_0, q_0) \geq 0$ for all $(x_0, q_0) \in \mathcal{Y}_0$, which corresponds to condition (4).
- Condition (11) states $B_k(x, q_k) < 0$ for all $(x, q_k) \in \mathcal{Y}_u$, which corresponds to condition (5).
- Condition (12) states $B_k(x, q) \geq 0 \Rightarrow B_k(x', q') \geq 0$ for all $(x', q') \in F(x, q)$, which corresponds to condition (6).

By Corollary (2.2), any trajectory of the product system $\mathfrak{S} \times \mathcal{A}$ cannot visit $\mathcal{Y}_u$.

Now we establish the connection to automaton runs. For any word $w = \mathfrak{L}(\tau) \in \mathfrak{L}(\mathfrak{S})$ where $\tau = \langle x_0, x_1, \dots \rangle$ is a trajectory of $\mathfrak{S}$, the corresponding run r of w on $\mathcal{A}$ is a sequence $r = \langle (r_0, w_0, r_1), (r_1, w_1, r_2), \dots \rangle$ where $r_0 = q_0$. The sequence of product states $\langle (x_0, r_0), (x_1, r_1), \dots \rangle$ forms a trajectory in $\mathfrak{S} \times \mathcal{A}$. Since this trajectory cannot visit $\mathcal{Y}_u = \{(x, q_k) \mid x \in \mathcal{X}\}$, we have $r_i \neq q_k$ for all i . Therefore, runs of all words in $\mathfrak{L}(\mathfrak{S})$ cannot visit state q_k . $\square$

THEOREM A.2. *Given a system $\mathfrak{S} = (\mathcal{X}, \mathcal{X}_0, f)$, an NBA $\mathcal{A} = (\Sigma, Q, q_0, \delta, Acc)$, and $(k, l) \in Acc^{pair}$, if a transition persistence certificate $B_{k,l}$ can be found, then it can be guaranteed that all runs of the words in $\mathfrak{L}(\mathfrak{S})$ make accepting transitions from q_k to q_l happen finitely often.*

PROOF. We define $\mathcal{X}_{k,l} = \{x \mid (q_k, \mathfrak{L}(x), q_l) \in Acc^{k,l}\}$ as the set of states where accepting transitions from q_k to q_l can occur.

Consider any trajectory $\tau = \langle x_0, x_1, \dots \rangle \in \mathfrak{T}(\mathfrak{S})$ and its corresponding word $w = \mathfrak{L}(\tau)$. Let r be a run of w on $\mathcal{A}$ starting from q_0 . An accepting transition from q_k to q_l can occur at step i only if $(q_k, \mathfrak{L}(x_i), q_l) \in Acc^{k,l}$, which is equivalent to $x_i \in \mathcal{X}_{k,l}$. Therefore, the number of accepting transitions from q_k to q_l is bounded by the number of times the trajectory visits $\mathcal{X}_{k,l}$.

By Definition 3.3, we have:

- $B_{k,l}(x_0) \geq 0$ by condition (13).
- $B_{k,l}(x_i) \geq 0$ for all i by condition (14), since $B_{k,l}$ remains non-negative along the trajectory.
- $B_{k,l}(x_i) \geq B_{k,l}(x_{i+1}) + I_{k,l}(x_i)\epsilon$ by condition (15), where $I_{k,l}(x_i) = 1$ if $x_i \in \mathcal{X}_{k,l}$ and 0 otherwise.

Suppose the trajectory visits $\mathcal{X}_{k,l}$ at indices $i_1, i_2, \dots, i_m$ where m is the number of times accepting transitions occur. At each visit, $B_{k,l}$ decreases by at least ϵ :

$$B_{k,l}(x_0) \geq B_{k,l}(x_{i_1}) + \epsilon \geq B_{k,l}(x_{i_2}) + 2\epsilon \geq \dots \geq B_{k,l}(x_{i_m}) + m\epsilon. \quad (40)$$

Since $B_{k,l}(x_{i_m}) \geq 0$ by condition (14), we have $B_{k,l}(x_0) \geq m\epsilon$, which gives $m \leq \frac{B_{k,l}(x_0)}{\epsilon}$. Therefore, the number of accepting transitions from q_k to q_l is bounded, ensuring they occur only finitely often. $\square$